

IRC Server Design

参考資料

- RFC 1459 - Internet Relay Chat Protocol - IRCの基本仕様
- RFC 2812 - Internet Relay Chat: Client Protocol - クライアントプロトコル詳細
- Qiita: 簡易サーバー例 - C言語でのソケットサーバー実装例

チャンネル名の先頭文字について

RFC 1459では `#` , `&` の2種類、RFC 2812では `#` , `&` , `+` , `!` の4種類が定義されている。

本プロジェクトでは課題要件によりS2S通信を実装しないため、以下の理由で `#` で始まるチャンネルのみを対応範囲とする：

- `&`（ローカルチャンネル）：S2S禁止のため全チャンネルが事実上ローカル。`#` との区別が不要
- `!`（安全チャンネル）：S2S環境での名前衝突防止用。単一サーバーでは不要
- `+`（モードなしチャンネル）：課題要件でMODE実装が必須のため、対応しない判断

なお、RFC 1459 Section 1.3には原文の欠落があり、`#` の記述が脱落している点に注意。

1. Purpose

このドキュメントは、IRCサーバ実装における全体設計・責務分割・主要クラスの役割を定義する。

本プロジェクトでは、複数人で並行実装するため、以下を明確にする。

- 各担当者の責務
- 各クラスの役割
- TCP接続状態とIRCユーザー状態の分離
- Channel / Client / ServerState の関係
- Network層とProtocol層の境界

詳細な関数プロトタイプは `docs/interface.md` に記載する。
実装状況・変更予定・未実装項目は `docs/roadmap.md` に記載する。

2. Design Policy

本プロジェクトでは、IRCサーバを以下の4つの層に分けて実装する。

Network / IO
Protocol / Command
Client / ServerState
Channel

2.1 完全ノンブロッキング & 単一 poll() の徹底

- Forkingは禁止する。
- すべてのI/O操作、つまり `listen` / `accept` / `read` / `write` はノンブロッキングで行う。
- 実際に `poll()` を呼ぶ場所は1箇所に限定する。
- `POLLIN` だけでなく、`POLLOUT` も適切に制御し、`send buffer`に未送信データがあるfdだけを書き込み監視対象にする。
- `Poller` を分離する場合も、`Server` から `Poller::wait()` を1箇所で呼ぶ構造にする。
- 各 `Connection` が個別に `poll()` / `select()` 等を呼ぶ設計は禁止する。

2.2 TCP接続とIRC状態の分離

- `Connection` はfd、ネットワークバッファ、送受信イベントの管理に専念する。
- `Client` はネットワークの詳細を知らず、IRC上のユーザー状態の管理に専念する。
- `Connection` と `Client` はfdで紐づくが、責務は明確に分離する。

2.3 状態とロジックの整理

- `Channel` はチャンネル内の状態を管理する。
- `ServerState` はサーバ全体の辞書を集中管理する。
- `Parser` は文字列をIRCメッセージ構造に変換する。
- `CommandDispatcher` はコマンド処理の入口を担当する。
- `ReplyBuilder` は返信文字列の生成に責務を限定する。
- Channel operator は、1人のユーザーが複数チャンネルで異なる権限を持つIRCの仕様に基づき、`Client` ではなく `Channel` が管理する。

3. Layer Overview

3.1 Network / IO Layer

担当: A

主なクラス

- `Server`
- `Connection`

必要に応じて分離するクラス

- `Poller`
- `ConnectionManager`

責務

- listen socket作成
- accept (ノンブロッキング)
- poll loop (`POLLIN` / `POLLOUT` のイベント待機と振り分け)
- fd監視・状態管理
- non-blocking read / write
- recv buffer / send buffer管理
- complete line (`\r\n` 単位) の切り出し
- 安全な disconnect 処理

補足

この層はIRCコマンドの意味、つまりセマンティクスを知らない。

知ってよいもの:

- `fd`
- `socket`
- `pollfd`
- `POLLIN`
- `POLLOUT`
- `recv buffer`
- `send buffer`
- `complete line`
- `fcntl`

知らない方がよいもの:

- `PASS`
- `NICK`
- `USER`
- `JOIN`
- `PRIVMSG`
- `MODE`
- `Channel`
- `operator`

3.2 Protocol / Command Layer

担当: B

主なクラス

- `Message`
- `Parser`
- `CommandDispatcher`
- `ReplyBuilder`

責務

- 1行の文字列を `Message` 構造体に変換する。
- IRC commandを判定する。
- `MODE` コマンドなどの複雑な引数、フラグ、対象パラメータを解析する。
- 各コマンドの処理入口を持つ。

- `ServerState` / `Client` / `Channel` を操作する。
- numeric reply / error reply / broadcast message を生成する。

補足

この層はソケット、バッファ、I/Oイベントなど、Networkの詳細をなるべく知らない。

`CommandDispatcher` は `Server` / `Connection` / `Poller` / `ConnectionManager` を直接操作しない。

コマンド処理結果は `CommandResult` として返し、送信処理は `Server` が担当する。

3.3 Client / ServerState Layer

担当: C1

主なクラス

- `Client`
- `ServerState`

必要に応じて分離するクラス

- `ClientRegistry`

責務

- IRC上の`Client`状態、認証状態、登録状態を管理する。
- fdから`Client`を引く。
- nickから`Client`を引く。
- nickの重複確認、および変更時の辞書更新を行う。
- 全体の辞書、つまり fd / nick / channel を保持する。
- `Client`の生成・削除のライフサイクルを管理する。
- `Client`削除時に、その`Client`が参加・招待・operator登録されている全`Channel`から参照を除去する。

補足

`ClientRegistry` は、`ServerState` が肥大化した場合に分離する補助クラスとする。初期実装では必須ではない。

3.4 Channel Layer

担当: C2

主なクラス

- `Channel`
- `ChannelModes`

必要に応じて分離するクラス

- `ChannelService`

責務

- channel名を管理する。
- 参加memberを管理する。
- channel operatorを管理する。
- topicを管理する。
- invited listを管理する。
- channel mode (`+i` , `+t` , `+k` , `+l`) を管理する。
- `JOIN` / `KICK` / `INVITE` / `TOPIC` / `MODE` の`Channel`側内部処理を担当する。
- 新規`Channel`作成直後、最初に参加した`Client`を`Channel Operator`にする。

補足

`ChannelService` は、`CommandDispatcher` が肥大化した場合に分離する補助クラスとする。初期実装では必須ではない。

4. Basic Processing Flow

IRCクライアントからメッセージを受け取って返信するまでの基本フローは以下。

```

poll()  [イベント待機]
↓
Server が fd イベントを受け取る
↓
[POLLINの場合]
Connection が recv() で recv buffer に読み込む
↓
EAGAIN / EWOULDBLOCK の場合は切断扱いにせず、次のイベントを待つ
↓
Connection が \r\n 単位で1行、つまり complete line を切り出す
↓
Parser が文字列を Message 構造体に変換する
↓
CommandDispatcher が command と引数を解析し、ServerState / Client / Channel を更新する
↓
ReplyBuilder が返信文字列、Numeric Reply、Broadcast用メッセージを生成する
↓
CommandDispatcher が送信対象fdと送信文字列を CommandResult に詰めて返す
↓
Server が CommandResult 内の OutgoingMessage を読み取る
↓
Server / Connection が対象fdの send buffer にデータを積む
↓
該当 fd の pollfd.events に POLLOUT フラグを登録する
↓
[次のpoll()でPOLLOUTが発生した場合]
Connection が send() で send buffer のデータをノンブロッキング送信する
↓
データ送信が完了し、send buffer が空になったら POLLOUT フラグを解除する

```

5. Connection and Client

本プロジェクトでは、`Connection` と `Client` を明確に分離する。

5.1 Connection

`Connection` はTCP接続そのもの、およびI/Oストリームを表す。

持つもの

- `fd`
- `recv buffer`
- `send buffer`

やること

- socketからノンブロッキングで読む。
- socketへノンブロッキングで書く。
- partial read / partial write、つまり不完全な読み書きへ対応する。
- complete line (`\r\n`) を抽出する。
- send bufferに残データがあるかを管理する。

やらないこと

- `NICK` の妥当性判定。
- `USER` の登録処理。
- `JOIN` の権限判定。
- `PRIVMSG` の配送先判断。
- `MODE` の意味解釈。

5.2 Client

`Client` はIRCプロトコル上のユーザー論理状態を表す。

持つもの

- `fd`
- `nick`
- `username`
- `realname`

- `host` (接続元ホスト)
- `PASS` 成功状態
- 登録完了状態

主なメソッド

- `getFd()`, `getNick()`, `getUsername()`, `getRealname()`, `getHost()` — 各フィールドのgetter
- `getFullPrefix()` — `nick!user@host` 形式を返す (RFC 1459 Section 2.3 準拠)
- `isRegistered()`, `isPassOk()`, `canRegister()`, `markRegistered()` — 状態管理

やること

- 自身のnickを保持する。
- 自身のusernameを保持する。
- 自身のrealnameを保持する。
- 自身のIRC登録状態・認証状態を保持する。

やらないこと

- socketから直接読む。
- socketへ直接書く。
- `recv` bufferを持つ。
- `send` bufferを持つ。

6. ServerState

`ServerState` はサーバ全体の状態辞書を管理する。

6.1 保持する主な辞書

```
fd      -> Client
nick    -> Client
channel -> Channel
```

6.2 主な責務

- Clientの追加・削除。
- fdからClientを検索する。
- nickからClientを検索する。
- nick重複チェックを行う。
- nick変更時に `nick -> Client` 辞書を更新する。
- Channelの取得・作成を行う。
- 空になったChannelを削除する。
- Client削除時に、全Channelから該当Clientへの参照を除去する。

6.3 Nick Update Rule

nick変更は必ず `ServerState::updateNick()` を通す。

NG:

```
client.setNick(newNick);
```

OK:

```
state.updateNick(client, newNick);
```

理由:

`Client` のnickだけを直接変更すると、`ServerState` 内の `nick -> Client` 辞書が古い状態のまま残り、不整合が発生するため。

6.4 Client Removal Rule

Client削除は必ず `ServerState::removeClient()` を通す。

`removeClient()` は、Client本体を削除する前に以下を行う。

- 参加中の全Channelからmember登録を削除する。
- operator集合から該当Clientを削除する。
- `invited list`から該当Clientを削除する。

- 空になったChannelを削除する。
- `fd -> Client` と `nick -> Client` の辞書を更新する。

理由:

`Channel` は `Client*` を所有せず参照だけを保持する。`Client`削除後に`Channel`側へ古い `Client*` が残ると、`PRIVMSG` / `MODE` / `KICK` などで不正参照が発生するため。

7. Channel and Operator

IRCにおける `operator` は、サーバ管理者、つまりIRC Operatorではなく、チャンネル内の権限ユーザー、つまりChannel Operatorを指す。

本プロジェクトでは、`operator`権限を `Client` ではなく `Channel` 側で集中管理する。

`Channel`

- `members` (参加中のClient一覧)
- `operators` (Operator権限を持つClient一覧)
- `invited` (招待されたClient一覧)
- `modes` (チャンネルのモード状態)

理由:

- 1人のClientは複数Channelに同時に参加できる。
- Channelごとにoperator権限の有無が異なる。
- そのため、`Client` 側に `isOperator` フラグを持たせると整合性が崩れる。

7.1 Operator Bootstrap

新しいChannelを作成した場合、最初にそのChannelへ参加したClientをChannel Operatorにする。

`JOIN #new`

- ↓
- `Channel #new` が存在しない
- ↓
- `Channel`を作成する
- ↓
- `JOIN`したClientを`members`へ追加する
- ↓
- 同じClientを`operators`へ追加する

理由:

Channel Operator専用コマンド（`KICK` / `INVITE` / `TOPIC` / `MODE`）を実行できる主体を初期化しないと、新規Channelでoperator権限を操作できなくなるため。

7.2 ChannelModes

`ChannelModes` は以下のmodeフラグを管理する。

- `+i` : invite only。招待されたユーザーのみ参加可能。
- `+t` : topic変更権限をoperatorのみに制限。
- `+k` : channel key。チャンネル参加にパスワードを要求。
- `+l` : user limit。最大参加人数制限。

`+o` は `ChannelModes` の状態フラグではなく、`Channel::_operators` の集合で管理する。

理由:

`+o` はチャンネル自体の属性ではなく、特定のClientに対する「operator権限の付与・剥奪」操作であるため。

7.3 Channel Validation Responsibility

`JOIN` / `KICK` / `INVITE` / `TOPIC` / `MODE` の権限判定は、最終的には `CommandDispatcher` が返信生成まで含めて制御する。

ただし、Channel側の状態に依存する判定は `Channel` / `ChannelModes`、または必要に応じて `ChannelService` に集約する。

主な判定:

- `JOIN` : invite-only、channel key、user limit、既参加かどうか。
- `KICK` : 実行者がChannel Operatorか、対象ClientがChannel memberか。
- `INVITE` : 実行者がChannel Operatorか、対象Clientが既にChannel memberでないか。
- `TOPIC` : topic表示、topic変更、`+t` 時のoperator制限。
- `MODE` : 実行者がChannel Operatorか、`+i` / `+t` / `+k` / `+l` / `+o` の引数が妥当か。

TOPIC #channel のような表示系コマンドと、MODE #channel のような照会系コマンドは、状態変更ではなく現在状態をNumeric Replyとして返す。

8. Class Responsibility Summary

Layer	Class	Status	Responsibility
Network / IO	Server	必須	サーバ起動、メインループ全体の統括、イベント振り分け
Network / IO	Connection	必須	fd、recv buffer、send buffer の保持、ノンブロッキング送受信の実行
Network / IO	Poller	必要に応じて分離	pollfd 配列の管理、POLLIN / POLLOUT のイベント制御・切り替え
Network / IO	ConnectionManager	必要に応じて分離	fd -> Connection 辞書の管理、安全な接続終了処理
Protocol / Command	Message	必須	パースされたIRCメッセージの構造体
Protocol / Command	Parser	必須	受信した1行の文字列を解析し、Message 構造体へ変換
Protocol / Command	CommandDispatcher	必須	コマンド名の判定と、対応する各コマンド処理ロジックへのルーティング
Protocol / Command	ReplyBuilder	必須	IRC返信文字列、Numeric Reply、Error、Broadcast用メッセージの生成
Protocol / Command	CommandResult	必須	コマンド処理結果。送信対象fdと送信文字列、切断要求をServerへ返すための境界オブジェクト
Client / ServerState	Client	必須	ユーザー固有の情報、認証状態、登録状態の管理
Client / ServerState	ServerState	必須	fd、nick、channel 各種辞書の集中管理と不整合の防止
Client / ServerState	ClientRegistry	必要に応じて分離	Client辞書分離用
Channel	Channel	必須	チャンネルの内部状態、参加者、operator、topic等の管理
Channel	ChannelModes	必須	チャンネルモード（+i、+t、+k、+l）の状態管理
Channel	ChannelService	必要に応じて分離	Channel操作が肥大化した場合の補助ロジック

9. Team Responsibility

担当者	担当範囲	必須ファイル	必要に応じて分離
A	Network / IO	Server、Connection	Poller、ConnectionManager
B	Protocol / Command	Message、Parser、CommandDispatcher、ReplyBuilder	なし
C1	Client / ServerState	Client、ServerState	ClientRegistry
C2	Channel	Channel、ChannelModes	ChannelService

10. Current Mock Implementation Policy

A担当の既存モック実装（Server.hpp/cpp）を出発点として採用する。

ただし、現状のモックは1つの Server クラスに以下の責務が混在している。

```
Server
Poller
Connection
ConnectionManager
```

そのため、短期的には動作確認用として利用し、段階的かつ速やかに責務を分離する。

10.1 Current Mock Status

Server.hpp/cpp	
└ Serverの責務	実装済み
└ Pollerの責務	Server内に仮実装（POLLINのみ、POLLOUT未対応）
└ Connectionのrecv側責務	Server内に仮実装（切断時の安全ループ未対応）
└ Connectionのsend側責務	即sendのみ（partial write未対応、要変更）
└ ConnectionManagerの責務	Server内に一部仮実装

10.2 Planned Changes

対象項目	現状のモック実装	変更予定
processMessage()	echo replyを即座に send() している	担当Bの Parser / CommandDispatcher へ文字列を渡す入口に変更する
send() 処理	processMessage() 内で直接ソケット送信	queueSend(fd, msg) 方式に変更し、send buffer に積んだ後、POLLOUT イベント経由で非同期送信する
_recvBuffers	Server クラスが map<int, string> で一括保持	各 Connection インスタンス内部へカプセル化する
_pollfds	Server クラスが std::vector<struct pollfd> を直接操作	必要に応じて Poller クラス内に隠蔽し、イベントの追加・削除・更新メソッドを提供する
ノンブロッキング化	#ifdef __APPLE__ の場合のみ fcntl を実行	OS環境を問わず、listen socketおよびacceptしたclient socketの生成直後に fcntl(fd, F_SETFL, O_NONBLOCK) を実行する
recv <= 0 の扱い	即座にその場で切断・削除を行っている	EAGAIN / EWOULDBLOCK は正常系として扱い、切断しない
ループ中の erase	receiveData(idx) 内で _pollfds.erase() を実行	ループ中の削除でインデックスが壊れないよう、削除対象fdをマークするか、disconnectClient(fd) 内で安全に制御する

11. MVP Definition

まずは以下の基本機能が正常に動作する状態を最優先のゴール、つまりMVPとする。

- サーバが正常に起動・待機できること。
- 複数クライアントがハングアップせずに同時接続・維持できること。
- PASS / NICK / USER の一連のフローでユーザー登録ができること。
- JOIN コマンドでチャンネルに入れること。
- PRIVMSG コマンドでチャンネル内および個人の会話ができること。
- PING / PONG で接続維持ができること。
- クライアントが切断しても、サーバがクラッシュ・ハングせず稼働し続けること。

MVPの段階では後回し、ただし最終提出までには必須実装

- KICK
- INVITE
- TOPIC
- MODE の各種フラグ (+i, +t, +k, +o, +l) および複合パラメータへの対応
- Numeric Replyのメッセージフォーマットの精度調整

実装肥大化時に後から分離する候補

- Poller
- ConnectionManager
- ClientRegistry
- ChannelService

12. Notes

- 詳細な関数プロトタイプは dev_docs/interface.md に記載する。
- 実装状況は dev_docs/roadmap.md に記載する。
- 課題PDF（要件）由来のメモは dev_docs/notes/subject.md に記載する。
- チーム内の決定事項は dev_docs/notes/meeting-log.md に記載する。